BAB I PENDAHULUAN

Bab ini berisi sejumlah penjelasan awal mengenai topik yang dibahas pada tugas akhir ini, terdiri dari latar belakang, rumusan masalah, tujuan, batasan masalah, metodologi, dan sistematika penulisan.

Latar Belakang

Sejak dahulu kala, informasi merupakan salah satu kebutuhan manusia yang paling mendasar. Salah satu penyebab mengapa informasi ini sangatlah penting karena manusia memiliki kodrat sebagai makhluk sosial. Pada dasarnya manusia memerlukan informasi untuk menentukan tindakan apa yang harus dilakukan. Informasi juga merupakan hal yang vital di dalam proses komunikasi.

Data merupakan informasi yang belum diolah, artinya data ini hanya merupakan kumpulan fakta yang dapat berupa angka, karakter, simbol, gambar, dan lain sebagainya. Data ini merupakan deskripsi dari sesuatu dan kejadian yang kita hadapi. Data yang telah diproses menjadi sesuatu yang memiliki makna maka akan disebut sebagai informasi. Kebanyakan sumber informasi menyediakan informasi masih dalam bentuk data, dan kemudian akan diolah di otak manusia yang menerimanya sehingga menjadi informasi yang bermanfaat.

Data maupun informasi akan disimpan atau diarsip sehingga dapat digunakan kembali di kemudian hari. Penyimpanan data atau informasi dapat menggunakan media penyimpanan, misalnya *notes, harddisk, flashdisk,* CD/DVD, maupun dengan menggunakan media penyimpanan lainnya.

Seiring dengan perkembangan teknologi, pertumbuhan jumlah data maupun informasi berkembang lebih pesat dibandingkan dengan kapasitas media penyimpanan. Karena keterbatasan kapasitas penyimpanan tersebut, maka muncul gagasan untuk menyimpan data atau informasi tersebut dengan lebih efisien. Dengan menyimpan data secara lebih efisien, maka manusia dapat menghemat ruang penyimpanan sehingga jumlah data yang disimpan bisa lebih banyak. Salah

satu solusi untuk menyimpan data secara lebih efisien ialah dengan menggunakan teknik kompresi data.

Ide dasar dari pemampatan data ialah banyaknya redundansi (pengulangan) pada representasi data. Pada dasarnya, kompresi data atau pemampatan data ialah memampatkan ukuran data sehingga menjadi lebih kecil dibandingkan sebelumnya. Hal ini bisa dilakukan karena ukuran data akan di-*encode* menjadi bit-bit yang lebih kecil dengan menggunakan metode tertentu.

Dua buah metode pemampatan data yang sering digunakan ialah Huffman Coding dan LZW (Lempel Ziv Welch) Coding. Pada metode pemampatan data Huffman, setiap karakter akan di-*encode* berdasarkan jumlah kemunculan masing-masing karakter di dalam suatu file teks, sehingga karakter yang sering muncul akan di-*encode* menjadi lebih pendek, dan begitupun sebaliknya karakter yang tidak sering muncul akan di-*encode* menjadi lebih panjang.

Sedangkan pada LZW Coding menggunakan struktur data yaitu kamus untuk proses *encode* karakter pada file teksnya. Setiap karakter atau frasa yang belum ada di dalam kamus akan dimasukkan ke dalam kamus sehingga kumpulan-kumpulan karakter yang telah berada di dalam kamus bisa di-*encode* menjadi karakter tertentu dengan panjang yang lebih kecil. Hasil akhir dari kedua algoritma ini ialah panjang file output akan memiliki ukuran yang lebih kecil dibandingkan file inputnya.

Untuk mengoptimasi algoritma pemampatan data, digunakan suatu metode transformasi yang disebut transformasi Burrows Wheeler. Transformasi Burrows Wheeler mentransformasikan isi dari file teks berdasarkan permutasi seluruh kemungkinan rotasi dari karakter-karakter di dalam file teks. Hasilnya ialah file teks yang akan dimampatkan mengalami perubahan bentuk, dimana bagian-bagian yang memiliki kemiripan akan ditempatkan secara berdekatan.

Rumusan Masalah

Ide yang mendasari pengerjaan tugas akhir ini ialah untuk melakukan perbandingan kinerja hasil optimasi terhadap dua buah algoritma kompresi data yang cukup popular, yaitu algoritma Huffman dan algoritma Lempel Ziv Welch (LZW). Optimasi algoritma diharapkan dapat menghasilkan data hasil kompresi yang lebih termampatkan.

Secara rinci, tahapan dari rumusan masalah di atas ialah:

1. Bagaimana melakukan optimasi terhadap kinerja algoritma Huffman dengan menggunakan metode transformasi Burrows Wheeler.
2. Bagaimana melakukan optimasi terhadap kinerja algoritma LZW dengan menggunakan metode transformasi Burrows Wheeler.
3. Bagaimana melakukan pengukuran tingkat kinerja algoritma Huffman pada saat pemampatan data baik sebelum maupun setelah mengalami optimisasi.
4. Bagaimana melakukan pengukuran tingkat kinerja algoritma LZW pada saat pemampatan data baik sebelum maupun setelah mengalami optimisasi.
5. Bagaimana membandingkan kinerja kedua algoritma tersebut sebelum maupun setelah mengalami optimisasi.

Tujuan

Tujuan yang ingin dicapai dari pengerjaan tugas akhir ini ialah membandingkan kinerja algoritma LZW dan Huffman sebelum dan sesudah dioptimasi sehingga dapat diketahui manakah diantara kedua metode tersebut yang menghasilkan rasio kompresi yang lebih baik.

Adapun tahapan untuk mencapai tujuan tersebut ialah:

1. Mempelajari dan memahami algoritma kompresi data Huffman, algoritma LZW, dan transformasi Burrows Wheeler.

2. Melakukan optimasi terhadap kinerja algoritma Huffman dan LZW dengan menggunakan transformasi Burrows Wheeler.
3. Melakukan implementasi algoritma Huffman, LZW, dan transformasi Burrows Wheeler untuk melakukan pengujian kinerja algoritma.
4. Membandingkan hasil pengujian kinerja kedua algoritma sebelum maupun sesudah dioptimasi dengan transformasi Burrows Wheeler.

Batasan Masalah

Dalam melaksanakan tugas akhir ini, masalah akan dibatasi pada:

1. Algoritma Huffman yang digunakan ialah algoritma dasar Huffman, bukan algoritma *Adaptive Huffman*.
2. Optimasi algoritma dilakukan dengan menerapkan transformasi Burrows Wheeler pada berkas yang akan dimampatkan.
3. Berkas yang dimampatkan adalah berkas teks ASCII. Berkas teks disini didefinisikan sebagai berkas teks yang dapat dibuka dan dibaca dengan menggunakan kakas penyunting teks *Notepad.*

Metodologi

Dalam mengerjakan tugas akhir ini, ada beberapa langkah yang dilakukan, yaitu:

1. Studi Literatur

Pada tahap ini dilakukan pembelajaran mengenai teori dasar yang digunakan pada tugas akhir ini, antara lain tentang teori kompresi data, algoritma Huffman, algoritma LZW, dan transformasi Burrows Wheeler. Literatur yang digunakan antara lain bahan dari situs internet, makalah ilmiah, maupun tugas akhir yang relevan dengan topik kompresi data.

2. Analisis

Pada tahap ini dilakukan analisis mengenai optimasi yang dilakukan, antara lain hipotesis optimasi dengan menggunakan transformasi Burrows Wheeler dapat meningkatkan kinerja algoritma kompresi Huffman maupun LZW. Pada tahap ini

juga dilakukan analisis terhadap perangkat lunak yang dibangun untuk melakukan pengujian terhadap kinerja kedua algoritma sebelum dan sesudah dioptimasi.

3. Perancangan dan Implementasi

Pada tahap ini dilakukan perancangan serta implementasi algoritma kompresi dalam bentuk perangkat lunak. Perangkat lunak yang dibangun berdasarkan hasil analisis yang telah dilakukan sebelumnya. Perangkat lunak ini berguna untuk membantu mengukur kinerja kedua algoritma sebelum maupun sesudah dioptimasi dengan transformasi Burrows Wheeler.

4. Pengujian dan Evaluasi

5. Kesimpulan dan Saran

Sistematika Penulisan

BAB II TINJAUAN PUSTAKA

Bab ini berisi pembahasan mengenai teori-teori hasil studi literatur yang melandasi pembuatan tugas akhir ini. Adapun hal-hal yang dibahas terdiri dari teori data, teori pemampatan data, algoritma Lempel Ziv Welch, algoritma Huffman, algoritma transformasi Burrows Wheeler, serta kajian yang terkait dengan tugas akhir ini.

Teori Data dan Informasi

Pada dasarnya data dapat diartikan sebagai representasi dari fakta. Awalnya, data hanyalah merupakan kumpulan fakta yang belum memiliki makna. Untuk memperoleh makna dari suatu data, data tersebut harus mengalami pemrosesan, sehingga akan berubah menjadi informasi.

Dalam kamus standar informasi dapat diartikan sebagai: (1) pengetahuan yang diperoleh dari pembelajaran, pengalaman, atau instruksi; (2) pengetahuan dari situasi atau event tertentu; (3) koleksi fakta atau data; (4) komunikasi dari suatu pengetahuan. Dalam keseharian, bentuk informasi yang paling banyak dikirimkan dan diterima adalah informasi dalam bentuk teks, suara, dan gambar.

Teori informasi yang dikembangkan oleh Claude E. Shannon mempelajari aspek kuantitatif dari informasi. Dengan menggunakan teori ini, dapat dihitung banyaknya informasi yang terdapat dalam suatu data dengan angka yang tepat. Karena itulah didefinisikan unit dari informasi yang disebut dengan bit. Saat ini sudah menjadi hal yang lumrah untuk menyebutkan ukuran dari suatu informasi dalam satuan bit. Teori informasi banyak diaplikasikan di berbagai bidang, diantaranya pemrosesan bahasa alami, kriptografi, komputasi kuantum, inferensi statistik, dan lain-lain.

Dalam ilmu komputer, data digunakan sebagai informasi yang dibutuhkan oleh komputer agar dapat menjalankan fungsinya. Beberapa penggunaan data oleh komputer antara lain:

1. Untuk menjalankan program

2. Untuk menyimpan program atau konfigurasi sistem
3. Untuk menyimpan fakta yang dibutuhkan oleh pengguna

Di dalam komputer, data disimpan dalam format *binary* dan direpresentasikan oleh kumpulan angka 0 dan 1. Setiap angka 0 maupun 1 menghabiskan ruang sebesar 1 bit di dalam media penyimpanan.

Jumlah data yang terus bertambah tidak diimbangi dengan peningkatan kapasitas media penyimpanan data. Oleh karena itu, manusia mencari cara agar dapat menyimpan data secara lebih efektif untuk memanfaatkan kapasitas media penyimpanan secara optimal. Salah satu gagasan yang muncul ialah bagaimana merepresentasikan data dalam bentuk yang lebih ringkas, sehingga data dapat disimpan dalam ukuran yang lebih kecil.

Teori Probabilitas

Probabilitas atau peluang sering diguakan untuk menganalisis kejadian acak. Teori ini merupakan pemodelan matematika dari kemungkinan terjadinya suatu kejadian. Dengan menggunakan teori probabilitas, dapat diprediksi hal-hal yang mungkin akan terjadi.

Dasar dari teori ini ialah ruang sampel, kejadian, dan titik sampel. Ruang sampel adalah himpunan dari semua hasil yang mungkin pada suatu percobaan atau kejadian. Kejadian adalah himpunan bagian dari ruang sampel yang ingin diketahui berapa probabilitasnya. Sedangkan titik sampel adalah anggota-anggota dari ruang sampel atau bisa juga disebut kemungkinan-kemungkinan yang muncul.

Probabilitas suatu kejadian X adalah sebagai berikut:

$$P(X) = \frac{Jumlah\ \ titik\ \ sampel\ \ termasuk\ \ X}{Jumlah\ \ ruang\ \ sampel}$$

Nilai probabilitas berada di rentang 0 sampai 1. Nilai 0 menyatakan kejadian tidak mungkin terjadi, sedangkan nilai 1 menyatakan kejadian yang pasti terjadi.

Jumlah dari seluruh probabilitas dalam ruang sampel adalah 1, sehingga probabilitas kejadian tersebut tidak terjadi ialah satu dikurangi probabilitas kejadian tersebut terjadi.

Teori Pohon

Pohon di dalam teori graf adalah himpunan terbatas dari satu atau lebih titik (node) dan busur yang menghubungkan titik tersebut sehingga tidak membentuk graf siklik. Pada himpunan titik tersebut terdapat satu titik yang disebut akar (root).

Kasus khusus dari struktur pohon ialah pohon biner. Pada pohon biner, setiap titik dapat memiliki paling banyak dua cabang ataupun tidak memiliki cabang sama sekali.

Beberapa terminologi pada pohon antara lain:

1. Node, adalah titik yang merupakan anggota dari pohon
2. Root, adalah akar yang merupakan titik yang tidak memiliki parent
3. Child, adalah cabang dari suatu titik
4. Parent, apabila titik x merupakan cabang dari titik y, maka titik y dikatakan sebagai parent dari titik x
5. Sibling, apabila titik y memiliki cabang dari yaitu titik x dan titik z, maka titik x dan titik z merupakan sibling
6. Depth, adalah kedalaman atau tinggi dari pohon
7. N-ary, adalah pohon dengan setiap titik memiliki cabang maksimal sebesar n titik. Pada pohon binary berarti nilai n adalah dua

Teori String dan ASCII

Pengertian string di dalam ilmu komputer adalah sekuens dari simbol yang dipilih dari sebuah set alfabet. Sedangkan di dalam pemrograman komputer, sebuah string merupakan salah satu tipe data yang menyimpan nilai data, biasanya dalam bentuk byte. Pada metode pemampatan data yang akan dibangun, file input akan dibaca sebagai sebuah string.

Operasi yang akan digunakan pada metode pemampatan data ialah operasi konkatenasi. Konkatenasi ialah penggabungan dua atau lebih string. Contohnya string s ialah “Bear” dan string t ialah “Hug”. Maka konkatenasi dari st ialah “BearHug”, sedangkan konkatenasi dari ts ialah “HugBear”

ASCII adalah singkatan dari American Standard Code for Information Interchange. ASCII adalah skema *encoding* yang digunakan pada karakter berdasarkan urutan pada alfabet bahasa Inggris. Kode ASCII akan merepresentasikan teks di dalam komputer.

Tabel ASCII bisa dilihat pada gambar berikut.

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

Source: www.LookupTables.com

Teori Pemampatan Data

Pemampatan data dapat diartikan sebagai proses mengubah aliran data masukan ke dalam aliran data lain yang lebih kecil. Pemampatan data, atau disebut juga kompresi data memiliki berbagai matode berdasarkan ide yang berbeda-beda. Proses pemampatan data menjadi populer dalam perkembangan ilmu komputer dikarenakan oleh hal-hal berikut:

1. Manusia memiliki kecenderungan untuk mengumpulkan data secara terus-menerus tanpa menghapus data sebelumnya. Hal ini tentunya menjadi masalah karena kapasitas media penyimpanan semakin lama semakin berkurang. Akibatnya, data baru yang akan disimpan tidak lagi memiliki tempat penyimpanan. Dengan menggunakan metode pemampatan data, jumlah data yang disimpan dalam media penyimpanan dapat ditingkatkan.
2. Waktu transfer data dapat dipercepat apabila data berukuran lebih kecil. Hal ini sangat bermanfaat, terutama karena pada era internet sekarang, kecepatan transfer data menjadi *constraint* yang sangat penting untuk para pengguna internet. Dengan menggunakan metode pemampatan data yang sesuai, data yang akan ditransfer dapat dikurangi ukurannya dan menyebabkan waktu transfer data menjadi berkurang.

Prinsip dasar metode pemampatan data ialah dengan menyingkirkan bagian yang dianggap redundan dari aliran data masukan. Hal ini berdasarkan prinsip bahwa data memiliki struktur tertentu, dimana struktur ini dapat dimanipulasi untuk mendapatkan representasi data dalam ukuran yang lebih kecil. Karena struktur data dapat berubah antara berbagai jenis data, maka tidak ada algoritma pemampatan data yang bekerja secara optimal untuk seluruh jenis data.

Metode pemampatan data dapat dibagi menjadi dua bagian, yaitu pemampatan data lossless dan pemampatan data lossy. Pada pemampatan data lossy, data yang dimampatkan akan memiliki perbedaan dengan data asal karena pada metode ini sebagian informasi pada data asal akan dihilangkan. Metode pemampatan data lossy banyak digunakan untuk memampatkan data video, suara, dan gambar. Hal ini dikarenakan pada tipe data tersebut, tidak akan terlihat perbedaan yang signifikan apabila sebagian informasi dari tipe data tersebut dihilangkan. Namun lain halnya pada data teks, dimana perubahan informasi, bahkan hanya sebesar 1 bit, dapat mengakibatkan perbedaan yang signifikan antara data awal dan data hasil pemampatan. Karena itulah, pada pemampatan data teks sebaiknya menggunakan metode lossless, yaitu data asal sama dengan data hasil pemampatan.

Beberapa hal yang dapat digunakan untuk mengukur kinerja dari sebuah algoritma pemampatan data ialah sebagai berikut:

1. Rasio pemampatan data (compression ratio)

Adalah ukuran hasil pemampatan data dibagi dengan ukuran file semula. Semakin kecil nilai rasio pemampatan data, semakin efektif kinerja algoritma kompresi tersebut. Sebagai contoh, apabila nilai rasio 0,7 berarti algoritma menghasilkan data keluaran berukuran 70% dari ukuran semula.

$$R_o = \frac{Ukuran\ \ keluaran\ \ aliran\ \ data}{Ukuran\ \ masukan\ \ aliran\ \ data}$$

2. Faktor pemampatan data (compression factor)

Adalah inverse dari rasio pemampatan data. Apabila nilainya lebih besar dari satu, maka algoritma pemampatan data tersebut tidak layak untuk digunakan.. Karena apabila nilai lebih dari satu maka terjadi ekspansi data, dan nilai kurang dari satu maka terjadi pemampatan data.

$$F_o = \frac{Ukuran\ \ masukan\ \ aliran\ \ data}{Ukuran\ \ keluaran\ \ aliran\ \ data}$$

3. Kecepatan kompresi (compression speed)

Adalah perbandingan ukuran berkas file yang akan dimampatkan dengan waktu yang dibutuhkan untuk memampatkan berkas file tersebut. Apabila kecepatan kompresi kecil maka waktu yang dibutuhkan untuk memampatkan data menjadi lama sehingga algoritma pemampatan data tersebut tidak banyak digunakan.

$$\text{Kecepatan kompresi} = \frac{Ukuran\ \ file\ \ semula}{Waktu\ \ kompresi}$$

4. Kecepatan dekompresi (decompression speed)

Adalah perbandingan ukuran berkas file hasil dimampatkan dengan waktu yang dibutuhkan untuk mendekompresi berkas file tersebut menjadi bentuk file awal.

Algoritma pemampatan data yang baik haruslah memiliki kecepatan kompresi dan dekompresi yang saling berimbang.

$$\text{Kecepatan dekompresi} = \frac{Ukuran\ \ file\ \ hasil\ \ pemampatan}{Waktu\ \ dekompresi}$$

Secara umum, saat ini ada dua macam pendekatan untuk metode pemampatan data, yaitu pendekatan secara statistik dan pendekatan dengan menggunakan kamus. Sebelum muncul kedua pendekatan tersebut, manusia menggunakan metode intuitif. Metode intuitif ini sangatlah sederhana, yaitu menghilangkan karakter-karakter selain huruf alfabetis. Akibatnya tentu saja hasil pemampatan data menjadi lossy dan performansi dari pemampatan data tidaklah memuaskan.

Setelah itu pemampatan data mulai memanfaatkan redundansi pada representasi karakter ASCII. Representasi karakter dengan ASCII memiliki redundansi yang besar, karena ASCII mengkodekan setiap karakter dengan panjang yang tetap. Dengan memanfaatkan redundansi tersebut, muncul gagasan untuk melakukan pengkodean karakter dengan panjang bit variatif, yang dikenal dengan sebutan *varibale size code*.

Untuk mendapatkan kinerja yang baik dari pengkodean bit dengan panjang variatif, maka diperlukan pengetahuan tentang jumlah kemunculan setiap karakter dalam suatu berkas file, agar setiap karakter dapat direpresentasikan dengan panjang kode yang berbeda. Karakter yang memiliki tingkat kemunculan tinggi direpresentasikan dengan kode yang lebih pendek, sedangkan karakter yang memiliki tingkat kemunculan rendah direpresentasikan dengan kode yang lebih panjang. Itulah mengapa pendekatan statistik digunakan, yaitu untuk menghitung kemunculan setiap karakter pada file. Salah satu pendekatan statistik yang populer ialah metode pemampatan data Huffman.

Karena pendekatan secara statistik sangat bergantung kepada permodelan statistik yang dibuat, maka manusia menciptakan pendekatan lain, yaitu pendekatan dengan menggunakan kamus. Pendekatan dengan menggunakan kamus tidak bergantung pada model statistik dan tidak menggunakan kode dengan panjang

variatif (variable size code). Pada pendekatan ini yang dilakukan ialah membangun sebuah kamus berdasarkan berkas masukan, dan kamus ini berguna untuk melakukan encoding string masukan menjadi token. Kamus ini dapat bersifat statis, yaitu kamus mengizinkan penambahan simbol saja, maupun bersifat adaptif, yaitu kamus mengizinkan penambahan simbol maupun pengurangan simbol dari dalam kamus.

Langkah-langkah umum pada pemampatan data dengan menggunakan pendekatan kamus ialah:

1. Melakukan pembacaan teks
2. Membuat kamus berdasarkan kemunculan kata pada berkas
3. Mengkodekan setiap kata berdasarkan indeks lokasi kata pada kamus

Metode pemampatan data Huffman (Huffman Coding)

Huffman Coding merupakan salah satu algoritma dengan menggunakan metode statistik. Hal ini disebabkan karena huffman coding memampatkan data berdasarkan jumlah kemunculan kata dari setiap karakter. Huffman Coding merupakan salah satu metode pemampatan data yang sangat popular karena banyak digunakan sebagai basis pada berbagai program di berbagai *platform.* Huffman Coding memiliki kemiripan dengan metode pemampatan data Shannon-Fano (Shannon-Fano Coding). Perbedaannya ialah, Huffman Coding mengkonstruksi pohon kode dari bawah ke atas, atau dengan kata lain mengkodekan simbol dari kanan ke kiri, sedangkan pada Shannon-Fano Coding ialah kebalikannya, yaitu membuat pohon kode dari atas ke bawah, atau dengan kata lain mengkodekan simbol dari bit paling kiri hingga bit paling kanan. Metode Huffman Coding ini dikembangkan oleh D. Huffman pada tahun 1952, dan telah menjadi subjek yang sering digunakan pada penelitian yang berhubungan dengan pemampatan data.

Algoritma dasar dari Huffman Coding ialah menggabungkan dua karakter yang paling tidak sering muncul pada berkas, dan mengulang proses ini hingga tersisa

sebuah karakter saja. Pada saat penggabungan ini, dibentuk struktur pohon kode dan kode Huffman akan diperolah dari proses *labeling* pada pohon kode tersebut.

Secara rinci, algoritma Huffman Coding adalah sebagai berikut. Mula-mula dibangun sebuah list dari simbol alfabet dalam urutan menurun berdasarkan kemunculan simbol tersebut di dalam suatu file. Kemudian dibentuklah pohon, dengan sebuah simbol pada setiap daun dari bawah ke atas. Hal ini dilakukan dalam beberapa tahap, dimana pada setiap tahap dua simbol dengan probabilitas terkecil akan dipilih, ditambahkan ke bagian atas dari pohon, dihapus dari list, dan diganti dengan sebuah simbol bantu yang merepresentasikan kedua simbol tersebut. Ketika list tersebut sudah tereduksi menjadi sebuah simbol bantu, maka pohon tersebut sudah selesai dibentuk. Selanjutnya pohon tersebut akan ditelusuri untuk menentukan kode dari setiap simbol.

Contoh dari algoritma Huffman dapat dijabarkan sebagai berikut:

Diberikan 5 simbol a_1, a_2, a_3, a_4, dan a_5, dengan probabilitas masing-masing 0.4, 0.2, 0.2, 0.1, dan 0.1. Simbol-simbol tersebut akan dipasangkan dengan urutan sebagai berikut:

1. a_4 digabungkan dengan a_5 dan keduanya diganti dengan simbol bantu a_{45}, dengan probabilitas 0.2
2. Sekarang terdapat 4 simbol tersisa, a_1, dengan probabilitas 0.4, a_2, a_3, a_{45}, dengan probabilitas masing-masing 0.2. Dengan memilih acak a_3 dan a_{45}, kemudian digabungkan, dan diganti dengan simbol bantu a_{345}, dengan probabilitas 0.4.
3. Sekarang tinggal 3 simbol yang tersisa, a_1, a_2, dan a_{345}, dengan probabilitas 0.4, 0.2, dan 0.4. Dengan memilih secara random a_2 dan a_{345}, kemudiang digabungkan, dan diganti simbol bantu a_{2345} dengan probabilitas 0.6.
4. Akhirnya, digabungkan dua simbol tersisa yaitu a_1 dan a_{2345} dan diganti dengan simbol bantu a_{12345} dengan probabilitas 1.

Sekarang pohon sudah selesai dibentuk. Pohon yang dibentuk dapat dilihat pada gambar xxx.



Dalam proses pengkodean, untuk setiap busur diberikan bit 1 untuk sisi atas dan bit 0 untuk sisi bawah. Hasilnya adalah simbol a1 dikodekan sebagai 0, simbol a2 dikodekan sebagai 10, simbol a3 dikodekan sebagai 111, simbol a4 dikodekan sebagai 1101, dan simbol a5 dikodekan sebagai 1100.

Yang perlu diingat bahwa kode pada algoritma Huffman tidaklah unik. Perhatikan gambar xxx, pada gambar di kanan proses pembentukan pohon memiliki perbedaan dengan pembentukan pohon pada gambar xxx. Gambar tersebut menunjukkan bahwa kelima simbol tersebut digabungkan secara berbeda menghasilkan kode yang berbeda pula, yaitu 11, 01, 00, 101, dan 100. Akan tetapi hasil penggabungannya haruslah menyisakan sebuah karakter dengan angka probabilitas sama dengan satu.

Beberapa hal yang perlu diperhatikan pada algoritma pemampatan data Huffman:

1. Tidak ada ketentuan bagaimana mengurutkan simbol-simbol alfabet, namun sebaiknya pengurutan didasarkan pada jumlah kemunculan setiap simbol pada file.
2. Tidak ada ketentuan bagaimana proses labeling pada busur pohon, ada dua cara yang bisa digunakan yaitu, sisi atas busur diberi bit 0 dan sisi bawah busur diberi bit 1 ataupun sebaliknya. Yang penting ialah konsistensi pemberian label pada busur pohon.
3. Kode pada algoritma Huffman tidaklah unik.

4. Kode pada algoritma Huffman optimal jika tidak ada kode-kode lain dengan panjang berbeda yang memiliki tingkat rata-rata panjang bit/karakter yang lebih rendah. Artinya ialah apabila salah satu pengkodean pada suatu file teks menghasilkan nilai rata-rata 2.2 bit/karakter, maka bentuk pengkodean lainnya harus menghasilkan nilai rata-rata 2.2 bit/karakter.

Metode Pemampatan Data Lempel Ziv Welch (LZW)

Algoritma pemampatan data Lempel Ziv Welch adalah algoritma pemampatan data dengan menggunakan pendekatan kamus. Kamus ini digunakan untuk menyimpan string yang telah ditemukan sebelumnya. Algoritma yang ditemukan oleh Abraham Lempel, Jacob Ziv, dan Terry Welch ini dipublikasikan sebagai implementasi yang lebih baik dari algoritma LZ78.

Untuk memahami algoritma LZW secara lebih mendalam, pertama-tama akan dibahas perbandingan algoritma LZW dengan algoritma LZ78. Pada awalnya kamus pada algoritma LZ78 kosong dan ukurannya terbatas oleh memori yang dapat digunakan. Pemampatan data dengan menggunakan algoritma LZ78 ialah sebagai berikut:

1. Kamus diawali dengan karakter null yang memiliki indeks 0
2. Baca karakter berikutnya pada file input. Setiap string yang ditambahkan pada kamus akan diberi indeks 1, 2, dan seterusnya.
3. Saat karakter x dibaca dari file input, kamus mencari sebuah token yang berisi string x. Token bisa ditulis dalam bentuk (X,Y), dimana X berupa bilangan bulat yang menunjukkan penunjuk pada kamus, dan Y berupa string.
    a) Jika string tidak ditemukan, maka token (0,x) ditambahkan di dalam kamus.
    b) Jika string x ditemukan, misalnya pada posisi k, maka string berikutnya y dibaca, dan kamus mencari token yang berisi xy. Jika tidak ditemukan, maka token (k,y) ditambahkan di dalam kamus.

Sedangkan apabila karakter ditemukan, maka proses pembacaan berlanjut terus hingga ditemukan karakter yang tidak berada di dalam kamus.

4. Kemudian ulangi dari langkah kedua hingga seluruh isi file terbaca.

Sedangkan pada algoritma pemampatan data LZW, kamus tidak diinisialisasi dengan karakter null, melainkan telah diisi dengan 0-255 karakter ASCII atau 256 8-bit byte. Perbedaan yang lain ialah, token pada LZW hanya terdiri atas satu elemen, yaitu penunjuk pada kamus, karena kamus pada LZW sudah diinisialisasi sehingga karakter yang akan dibaca berikutnya pasti telah ada di dalam kamus. Prinsip utama algoritma ini ialah memampatkan karakter satu per satu dan mengakumulasikannya ke dalam string I Pemampatan data dengan algoritma LZW adalah sebagai berikut:

1. Inisialisasi kamus dengan 256 entri di dalam kamus
2. Baca karakter berikutnya pada file input.
3. Setiap karakter dikonkatenasi dengan string I.

a) Apabila I terus menerus ditemukan di dalam kamus, maka proses pembacaan terus berlanjut.
b) Apabila konkatenasi karakter dengan string I tidak ditemukan di dalam kamus, misalnya Ix, maka kamus menambahkan token baru di dalam kamus, yaitu penunjuk ke string I, menyimpan string Ix di dalam kamus, dan menginisialisasi string I ke karakter x.

4. Ulangi mulai langkah yang kedua hingga pembacaan file input selesai.

Untuk lebih memahami perbedaan algoritma LZ78 dan LZW, diberikan contoh pemampatan data pada string: “sir sid eastman easily”

Pemampatan data dengan algoritma LZ78 adalah sebagai berikut:

Kamus		Token
0	null	
1	“s”	(0,”s”)

2	“i”	(0,”i”)
3	“r”	(0,”r”)
4	“ “	(0,” “)
5	“si”	(1,”i”)
6	“d”	(0,”d”)
7	“ e”	(4,” e”)
8	“a”	(0,”a”)
9	“st”	(1,”t”)
10	“m”	(0,”m”)
11	“an”	(8,”n”)
12	“ ea”	(7,”a”)
13	“sil”	(5,”1”)
14	“y”	(0,”y”)

Pemampatan dengan algoritma LZW adalah sebagai berikut:

I	Ada di dalam kamus?	Entri baru	Keluaran
s	Y		
si	T	256-si	115(s)
i	Y		
ir	T	257-ir	105(i)
r	Y		
r” “	T	258-r” “	114(r)
“ “	Y		
“ “s	T	259-“ “s	32(“ “)
s	Y		
si	Y		
sid	T	260-sid	256(si)
d	Y		
d” “	T	261-d” “	100(d)

“ “	Y		
“ “e	T	262-“ “e	32(“ “)
e	Y		
ea	T	263-ea	101(e)
a	Y		
as	T	264-as	97(a)
s	Y		
st	T	265-st	115(s)
t	Y		
tm	T	266-tm	116(t)
m	Y		
ma	T	267-ma	109(m)
a	Y		
an	T	268-an	97(a)
n	Y		
n” “	T	269-n” “	110(n)
“ “	Y		
“ “e	Y		
“ “ea	T	270-“ “ea	262(“ “e)
a	Y		
as	Y		
asi	T	271-asi	264(as)
i	Y		
il	T	272-il	105(i)
l	Y		
ly	T	273-ly	108(l)
y	Y		121(y)

Transformasi Berkas

Transformasi berkas adalah proses melakukan modifikasi pada berkas sehingga mengubah berkas menjadi bentuk yang lebih mudah untuk dimampatkan. Transformasi berkas biasanya dilakukan sebelum memampatkan data agar bisa mendapatkan hasil pemampatan data yang lebih baik.

Ada 3 jenis transformasi yang seringkali digunakan dalam memproses berkas sebelum dimampatkan, yaitu:

1. Transformasi Burrows Wheeler

Transformasi Burrows Wheeler menggunakan matriks untuk menyusun permutasi kemunculan karakter dalam berkas, sehingga akhirnya pola repetisi yang tadinya terletak secara tersebar dalam berkas teks dapat dikumpulkan pada lokasi yang berdekatan.

2. Transformasi Star

Transformasi Star berdasarkan pada pendefinisian tanda yang unik unuk setiap karakter dan menggantikan karakter tersebut dengan karakter khusus, contohnya karakter *. Transformasi ini menggunakan kamus untuk melakukan transformasi pada berkas.

3. Transformasi LIPT (Length Index Preserving Transform)

Transformasi LIPT merupakan pengembangan dari transformasi Star. Transformasi LIPT menggunakan kamus dan memiliki kecepatan yang lebih baik dibandingkan transformasi Star. Namun sayangnya transformasi ini belum popular digunakan.

Pada pembuatan tugas akhir ini dipilih Transformasi Burrows Wheeler untuk melakukan praprosesing berkas, karena transformasi ini adalah metode yang paling sederhana serta tidak membutuhkan kamus tambahan untuk melakukan transformasi berkas. Selain itu transformasi Burrows Wheeler dipilih karena merupakan metode transformasi berkas yang paling popular dan paling sering digunakan dalam penelitian.

Transformasi Burrows Wheeler

Transformasi Burrows Wheeler adalah salah satu metode yang digunakan untuk melakukan transformasi pada berkas teks. Transformasi yang terjadi ialah menjadikan bagian-bagian yang banyak berulang pada teks menjadi berdekatan. Pada proses transformasi, berkas teks hasil transformasi akan menjadi lebih besar karena ditambahkan variabel lokasi baris pada hasil transformasinya. Bagian yang saling berulang akan diletakkan berdekatan sehingga menjadikan berkas teks tersebut lebih mudah untuk dimampatkan.

Cara kerja algoritma transformas Burrows Wheeler adalah sebagai berikut:

1. Membaca string x
2. Membuat tabel yang berisi semua kemungkinan rotasi dari string x
3. Mengurutkan kolom tabel secara alfabetis
4. Mengembalikan kolom terakhir dan nomor baris yang mengandung berkas awal sebagai berkas yang telah ditransformasi

Cara untuk mengembalikan berkas hasil transformasi adalah sebagai berikut:

1. Membuat sebuah tabel kosong
2. Membaca string yang ditransformasi y
3. Ulangi sebanyak panjang y:

a) Masukkan string y sebagai kolom sebelum kolom pertama
b) Lakukan pengurutan secara alfabetis

4. Jadikan baris yang bersesuaian dengan nomor baris yang terdapat pada berkas yang telah ditransformasi sebagai keluaran

Untuk lebih memahami transformasi Burrows Wheeler, diberikan contoh seperti pada gambar berikut:

Indeks	1	2	3	4	5	6
Char	B	A	N	A	N	A

Indeks	1	2	3	4	5	6
Char	B	A	N	A	N	A
Rotasi 1	A	N	A	N	A	B
Rotasi 2	N	A	N	A	B	A
Rotasi 3	A	N	A	B	A	N
Rotasi 4	N	A	B	A	N	A
Rotasi 5	A	B	A	N	A	N

	1	2	3	4	5	6
1	A	B	A	N	A	N
2	A	N	A	B	A	N
3	A	N	A	N	A	B
4	B	A	N	A	N	A
5	N	A	N	A	B	A
6	N	A	B	A	N	A

Hasil dari transformasi kata “BANANA” adalah NNBAAA, 4. Dimana 4 adalah nomor baris yang menyimpan berkas awal kata “BANANA” dan “NNBAAA” adalah kolom terakhir pada tabel setelah pengurutan secara alfabetis.